

Spotify Lab: Vectors, Inner Products, and Similar Songs

Code walkthrough — each cell's code alongside a plain-language explanation, for reference while working through `lab_01.ipynb`.

Setup

Load the data and split columns into word-count vs. metadata

cell id: b69a84fc

CODE

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

songs = pd.read_csv( "./data/song_vectors.csv.gz" )
vocab = pd.read_csv( "./data/vocabulary.csv" )
top_40 = pd.read_csv("./data/top_40_songs_by_genre.csv")

word_cols = vocab["word"].tolist() # Song count matrix
meta_cols = songs.columns.drop(word_cols).tolist() # Information about
tracks

print('Number of songs, number of words:', songs.shape)
print( 'Song metadata fields:', meta_cols[:8])
```

WHAT IT DOES

Loads the three source files: the 8000-song word-count table, the 800-word vocabulary list, and the Top-40-by-genre benchmark set. It then splits the song table's columns into two lists: **word_cols** (the 800 word-count columns, taken from the vocabulary file) and **meta_cols** (everything else — track ID, artist, genre, release year, etc.). The prints confirm the shape: 8000 songs by 816 total columns (800 words + 16 metadata fields).

Preview the metadata-only view

cell id: afce43ce

CODE

```
songs[meta_cols].head()
```

WHAT IT DOES

Shows the first five songs using only the metadata columns (no word counts), so you can see what information exists about each track — artist, title, release, genre, popularity measures — before working with the word-count math.

Selecting a Query Song

Build the song-by-word matrix X and pick a query song

cell id: fe0e4449

CODE

```
X = songs[word_cols].to_numpy(dtype=float)

query_idx = 0 # Let's query the first song
query = X[query_idx]

print( 'Track metadata: \n')
print(songs.loc[query_idx, meta_cols])

print( 'Track word counts: \n')
print( query )
```

WHAT IT DOES

Pulls just the word-count columns out as a plain NumPy array, X — this is the song-by-word matrix, where each row is one song's 800-length vector. It picks song index 0 (Gary Moore, “What Are We Here For”) as the **query** song — the one you're finding matches for — and prints both its metadata and its raw count vector, which is mostly zeros since any one song only uses a small slice of the 800-word vocabulary.

List the query song's actual (nonzero) words

cell id: 20e8d07d

CODE

```
# Let's see the counts for nonzero words

nonzero = np.flatnonzero(query)

pd.DataFrame({
    "word": np.array(word_cols)[nonzero],
    "count": query[nonzero].astype(int),
}).sort_values("count", ascending=False).head(15)
```

WHAT IT DOES

Makes the mostly-zero vector human-readable: **np.flatnonzero** finds the index positions where the query vector is nonzero, then builds a small table pairing each used word with its count and sorts it descending. This shows the query song's top words at a glance (e.g. “we” 27 times, “are” 10 times) instead of scrolling through 800 mostly-empty numbers.

Step 1: Dot Product

Score every song against the query with the raw dot product

cell id: 7f61e5bd

CODE

```
# Compute the dot product between every song vector and the query
vector.
dot_scores = X @ query

dot_results = songs[meta_cols].copy()
dot_results["dot"] = dot_scores
dot_results.sort_values("dot", ascending=False).head(10)
```

WHAT IT DOES

$X @ \text{query}$ is a matrix-vector multiply that computes the dot product between every one of the 8000 song vectors and the query vector in a single operation. The scores are attached as a new **dot** column and the top 10 songs are shown — these are the first-draft “similar songs” recommendations under the raw dot-product rule.

Visual check: does song length inflate the score?

cell id: 4e58e7b6

CODE

```
# Visual check: long songs can receive larger dot products.
top_dot = dot_results.sort_values("dot",
ascending=False).head(10).copy()
top_dot["label"] = (
top_dot["artist_name"].fillna("") + " - " +
top_dot["song_title"].fillna("")
)
colors = ["#e76f51" if index == query_idx else "#457b9d" for index in
top_dot.index]
plt.figure(figsize=(9, 5))
plt.barh(top_dot["label"], top_dot["dot"], color=colors)
for position, words in enumerate(top_dot["n_words"]):
plt.text(top_dot["dot"].iloc[position], position, f" {words:.0f} lyric
words", va="center", fontsize=8)
plt.gca().invert_yaxis()
plt.xlabel("dot-product score")
plt.title("Dot product: overlap and song length both matter")
plt.tight_layout()
plt.show()
```

WHAT IT DOES

Draws a horizontal bar chart of the top 10 dot-product matches, with each bar labeled by the song's total lyric word count (**n_words**). This is the chart's whole point: it exposes the weakness of the raw dot product — longer songs with more total words tend to score higher regardless of whether their content is genuinely similar, simply because more words means more chances to overlap on any given term.

Step 2: Centering and Covariance

Subtract each song's own mean before scoring

cell id: ea0987e0

CODE

```
query_centered = query - query.mean()
X_centered = X - X.mean(axis=1, keepdims=True)

centered_dot_scores = X_centered @ query_centered
covariance_scores = centered_dot_scores / X.shape[1]

cov_results = songs[meta_cols].copy()
cov_results["centered_dot"] = centered_dot_scores
cov_results["covariance"] = covariance_scores
cov_results.sort_values("covariance", ascending=False).head(10)
```

WHAT IT DOES

Subtracts each song's own average word count from every entry in its vector (**query - query.mean()**, and the same per-row for all of **X**). The dot product of these centered vectors is **centered_dot**; dividing by the number of words turns it into **covariance**. Centering changes the question from “do these songs share raw word mass” to “do these songs rise and fall together relative to their own baselines.”

Visual check: what covariance is rewarding

cell id: ece4d11e

CODE

```
# Visual check: centering moves each song's average word count to zero.
comparison_idx = np.argsort(covariance_scores)[-2]
comparison = X[comparison_idx]
used_words = (query > 0) | (comparison > 0)
both_above_average = (query > query.mean()) & (comparison >
comparison.mean())
plt.figure(figsize=(6, 6))
plt.scatter(query[used_words], comparison[used_words],
c=np.where(both_above_average[used_words], "#e76f51", "#457b9d"),
alpha=0.65)
plt.axvline(query.mean(), color="#555555", linestyle="--",
label="query mean")
plt.axhline(comparison.mean(), color="#555555", linestyle=":",
label="match mean")
plt.xlabel("query word count")
plt.ylabel("matching song word count")
plt.title("Covariance rewards words above both songs' averages")
plt.legend()
plt.tight_layout()
plt.show()
```

WHAT IT DOES

Scatter-plots the query song's word counts against one specific comparison song's word counts (the #2 match by covariance), restricted to words either song actually uses. Dashed/dotted lines mark each song's own mean, and points are colored orange when **both** songs are above their own average on that word — exactly the pairs covariance rewards.

Step 3: Correlation

Rescale covariance by each song's spread

cell id: 316baaf1

CODE

```
query_std = query.std()
X_std = X.std(axis=1)

denom = X_std * query_std
corr_scores = np.divide(
covariance_scores,
denom,
out=np.zeros_like(covariance_scores),
where=denom != 0,
)

corr_results = songs[meta_cols].copy()
corr_results["correlation"] = corr_scores
corr_results.sort_values("correlation", ascending=False).head(10)
```

WHAT IT DOES

Divides the covariance score by the product of each vector's standard deviation (**query.std()** and per-row **X.std(axis=1)**), producing a correlation coefficient bounded between -1 and 1. The **np.divide(..., where=denom != 0)** pattern is a safety measure: it returns 0 instead of crashing for any song whose word counts have zero variance, avoiding a division-by-zero error.

Visual check: covariance vs. correlation

cell id: 3558825a

CODE

```
# Visual check: correlation rescales covariance by each song's spread.
plt.figure(figsize=(7, 5))
plt.scatter(covariance_scores, corr_scores, s=10, alpha=0.25,
            color="#457b9d")
plt.axhline(0, color="#555555", linewidth=0.8)
plt.axvline(0, color="#555555", linewidth=0.8)
plt.scatter(covariance_scores[query_idx], corr_scores[query_idx],
            color="#e76f51", s=45, label="query song")
plt.xlabel("covariance with query")
plt.ylabel("correlation with query")
plt.title("Correlation changes the scale, and sometimes the ranking")
plt.legend()
plt.tight_layout()
plt.show()
```

WHAT IT DOES

Plots every song's covariance score against its correlation score. Since standardizing by spread can reorder songs relative to each other, this scatter shows how much the ranking shifts — points that move a lot along the y-axis relative to their x-axis position are songs whose rank changed after accounting for variance, not just average level.

Step 4: Cosine Similarity

Normalize by vector length instead of centering

cell id: 179a79f2

CODE

```
def cosine_scores(matrix, query_vector):
    numerators = matrix @ query_vector
    matrix_norms = np.linalg.norm(matrix, axis=1)
    query_norm = np.linalg.norm(query_vector)
    denominators = matrix_norms * query_norm
    return np.divide(
        numerators,
        denominators,
        out=np.zeros_like(numerators),
        where=denominators != 0,
    )

cos_scores = cosine_scores(X, query)

cosine_results = songs[meta_cols].copy()
cosine_results["cosine"] = cos_scores
cosine_results.sort_values("cosine", ascending=False).head(10)
```

WHAT IT DOES

Defines a reusable function computing cosine similarity: the raw dot product in the numerator, divided by the product of the two vectors' L2 norms (Euclidean lengths) in the denominator, with the same zero-safe **np.divide** pattern as Step 3. Note the key difference from correlation: cosine similarity normalizes raw vector length but does **not** subtract the mean first — there's no centering step. It's applied to the query song and the top 10 results are shown, ready to compare against the Step 1 dot-product ranking.

Step 5: TF-IDF Reweighting

Down-weight common words, up-weight rare/distinctive ones

cell id: 6d73cda3

CODE

```
# Term frequency: repeated uses of one word count less and less.
TF = np.log1p(X) # Compute TF

# Document frequency: how many songs use each word at least once.
N = X.shape[0] # Determine number of documents
df = (X > 0).sum(axis=0) # Compute document frequency
IDF = np.log(N / df) # Compute IDF

# IDF is one number per word, so NumPy broadcasts it across every song
row.
TFIDF = TF * IDF

print(pd.DataFrame({
    "word": word_cols,
    "document_frequency": df,
    "idf": IDF,
}).sort_values("idf", ascending=True).head(10))

print( pd.DataFrame({
    "word": word_cols,
    "document_frequency": df,
    "idf": IDF,
}).sort_values("idf", ascending=False).head(10) )
```

WHAT IT DOES

Builds a reweighted word-count matrix that fixes a different problem than length or centering: common words like “the” dominate raw counts while rare, more distinctive words get drowned out. **TF = $\log(1+x)$** compresses the effect of repetition (1→2 uses matters more than 20→21). **df** counts how many of the 8000 songs contain each word at least once, and **IDF = $\log(N/df)$** turns that into a weight that’s large for rare words, near zero for words in almost every song. **TF * IDF** (NumPy broadcasts the one-per-word IDF vector across every row) produces the final **TFIDF** matrix. The printed tables confirm it: lowest-IDF words are near-universal (“i”, “and”, “you”), while highest-IDF words are rare, often non-English fragments appearing in only a few dozen songs — much more distinctive matching signals.

Everything past this point in the notebook (TF-IDF-based rankings across all four similarity measures, applied to all 40 benchmark songs, plus the Part 2 genre-precision evaluation) is left for you to build by reusing these same patterns.